BCaaS : une plateforme métier inspirée des réseaux

Présenter un Business Core multitenant comme une pile de protocoles : couches, encapsulation, routage, contrôle et interopérabilité.

3e année génie informatique

Objectif pédagogique : concevoir une architecture microservice multitenant en s’appuyant sur les réflexes de l’ingénierie de protocoles.

Business Core as a Service

1. Le problème à résoudre

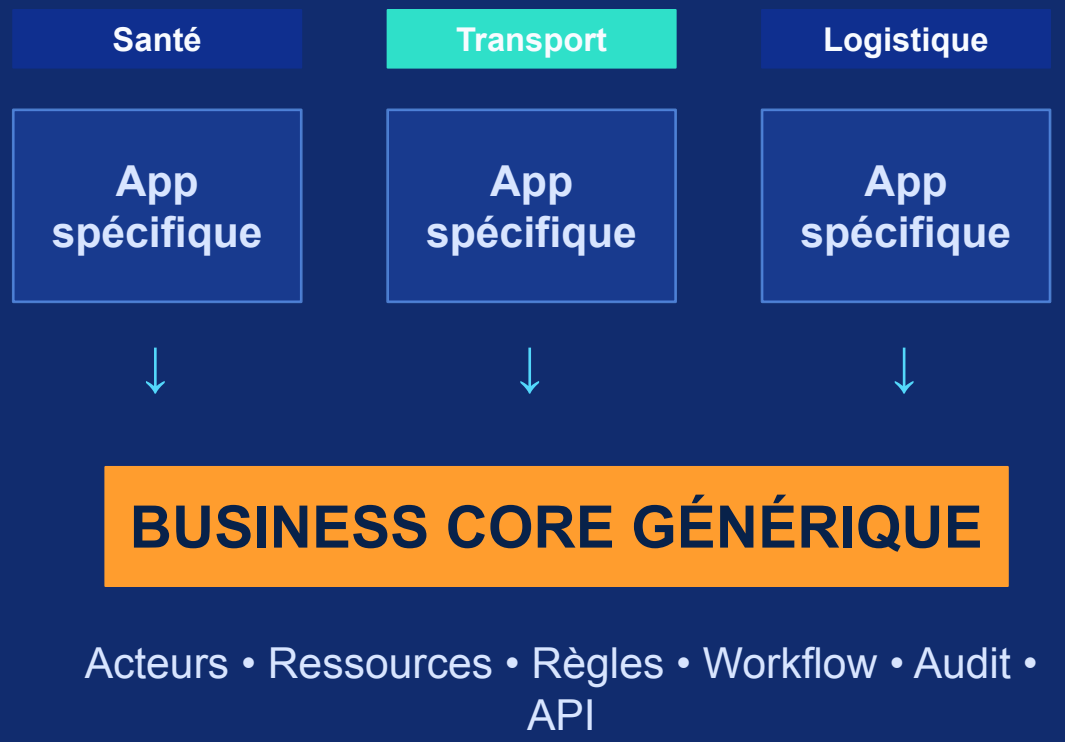
Du logiciel métier spécifique vers un socle réutilisable

Constat

- Chaque métier reconstruit les mêmes briques : acteurs, ressources, règles, workflow, audit, notifications.
- Les applications sont souvent couplées à un seul domaine et difficiles à faire évoluer.
- Le multitenant, la sécurité, la traçabilité et l'interopérabilité sont souvent ajoutés trop tard.

Vision

Construire un noyau métier générique, modulaire et configurable : le Business Core as a Service.



2. Pourquoi s’inspirer des réseaux ?

OSI et TCP/IP donnent une méthode, pas juste une métaphore

Séparer les responsabilités

Chaque couche sait quoi faire et quoi ignorer.

Encapsuler le contexte

Les métadonnées de transport sont séparées du payload métier.

Standardiser les échanges

Un protocole documenté facilite l’interopérabilité entre services.

Raisonner en flux

On conçoit les scénarios de bout en bout comme des conversations.

Penser système avant de coder

3. Analogie centrale : de la pile réseau à la pile métier

Une correspondance qui guide la conception

Adressage IP	Résolution du tenant, du domaine et du service cible
Transport TCP/UDP	Choix sync/async, retry, timeout, idempotence
Session	Contexte de processus, saga, corrélation
En-têtes	tenant_id, actor_id, trace_id, SLA, sécurité
QoS	Priorité d'une transaction, garanties et dégradation contrôlée

Le cœur métier devient une pile protocolaire

4. Pile protocolaire métier proposée

Un modèle en 5 couches pour structurer le Business Core

5. Business capabilities	Acteurs, ressources, opérations, workflow, documents, audit
4. Context & policy	Identité, permissions, règles, SLA, session métier, saga
3. Tenant & routing	Résolution du tenant, discovery, partitionnement, versioning
2. Transport & messaging	REST/gRPC, événements, retry, timeout, idempotence
1. Infrastructure	Base de données, cache, bus, stockage, observabilité technique

Principe : une couche offre des services à la couche supérieure et cache sa complexité à la couche inférieure.

Abstraction + encapsulation

5. Encapsulation : le “paquet métier”

Séparer le contexte d’acheminement du contenu métier

Header de transport	Header de contexte métier	Payload métier ex.: création de dossier, réservation, validation

Exemples de métadonnées : tenant_id • actor_id • role_scope • trace_id • locale • policy_level • correlation_id

Bénéfice : un service peut traiter la donnée tout en respectant les politiques communes sans connaître toute l’application.

Header ≠ payload

6. Deux idées réseau très utiles

Virtualisation multi-tenant et séparation control plane / data plane

A. Le multitenant comme réseau virtuel

Infrastructure partagée

Tenant A | **Tenant B** | **Tenant C**

Analogie : VLAN / namespace / overlay → isolation logique des clients sur une base technique commune.

B. Control plane / data plane

Control plane : configuration des règles, des tenants, des SLA, des workflows

↓

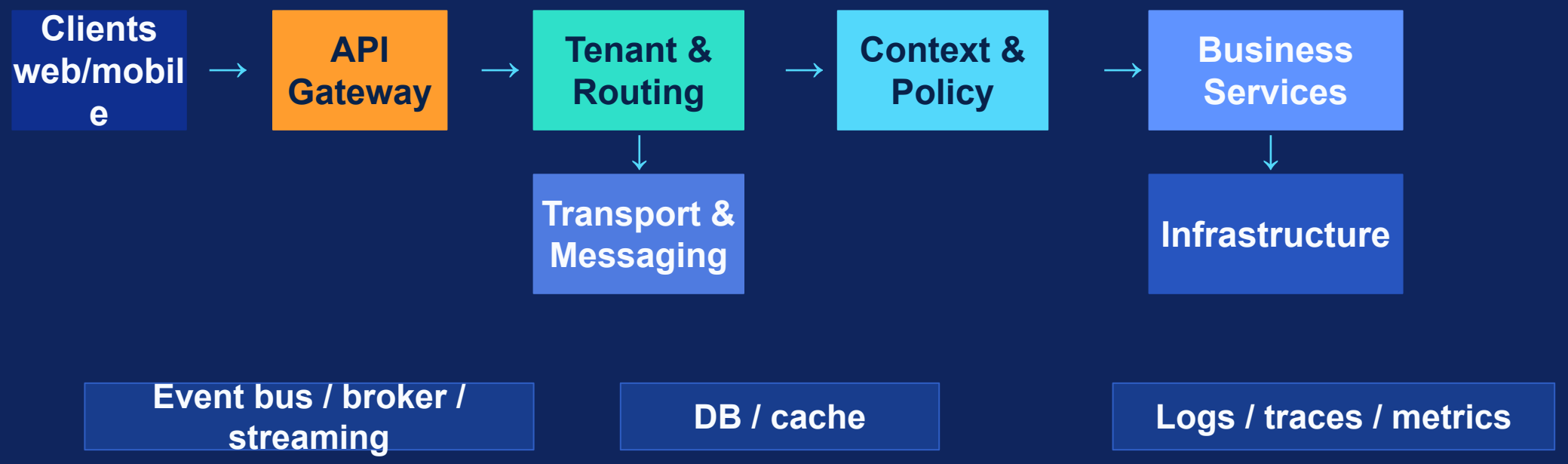
Data plane : exécution des transactions métier et circulation des événements

Idée clé : paramétrer le système sans redéployer son cœur opérationnel.

Gouverner séparément de l'exécution

7. Architecture microservice de référence

Une vue simple, lisible et cohérente avec la pile protocolaire



Ce diagramme montre une architecture où les préoccupations transverses ne polluent pas directement le code métier.

Vue logique d'ensemble

8. Exemple de flux bout-en-bout

Scénario : ouverture d'un dossier de service

1		2		3		4		5
Le client envoie la requête	→	**Le gateway authentifie et route**	→	**La couche policy enrichit le contexte**	→	**Le service métier valide et persiste**	→	**Un événement est publié**
payload + headers de transport		résout le tenant et la version API		permissions, corrélation, SLA		règles métier + transaction		notification, audit, analytics

Lecture réseau : requête → routage → enrichissement de contexte → traitement → propagation d' événements.

Toujours raisonner en séquence de messages

9. Ce que les étudiants doivent “protocoliser”

L’approche projet : définir des contrats avant d’implémenter

Message

types, formats, schémas, erreurs

États

automates, transitions, conditions

En-têtes

tenant, acteur, traçabilité, sécurité

Transport

sync vs async, retry, timeout

Sémantique

idempotence, compensation, ordre

Observation

logs, métriques, traces, KPI métier

Réflexe attendu : si deux services dialoguent, leur protocole doit être explicite, testable et versionnable.

Concevoir les contrats avant le code

10. Livrables attendus pour un mini-projet

Transformer les concepts en une architecture argumentée

- Un diagramme de couches inspiré d'OSI/TCP-IP
- Une architecture microservice avec composants et flux
- Un protocole d'échange pour un cas métier concret
- Une stratégie multitenant et d'isolation
- Une proposition d'observabilité et de QoS métier

Critères d' évaluation

Cohérence des couches • qualité des contrats • justification des choix • lisibilité de l'architecture

À retenir

Les réseaux apportent une méthode	couches, adressage, encapsulation, QoS, supervision.
Le Business Core devient une pile de protocoles	chaque microservice participe à un contrat global.
Votre mission d'ingénieur	modéliser les échanges avant d'implémenter les composants.

Question directrice : pour chaque instance du coeu de métuen, quel est le protocole de votre métier ?